

Plano de Containerização para Sistema de Comunicação SSE

Índice

1. [Visão Geral](#)
2. [Arquitetura de Containers](#)
3. [Serviços AWS Recomendados](#)
4. [Configuração dos Containers](#)
5. [Fluxo de Comunicação](#)
6. [Considerações de Segurança](#)
7. [Plano de Implementação](#)

Visão Geral

O projeto atual consiste em um sistema de comunicação em tempo real utilizando Server-Sent Events (SSE) para estabelecer um canal de comunicação entre o backend e o frontend. O sistema processa consultas, realiza buscas e fornece respostas estruturadas, com suporte para validação e análise de qualidade.

A containerização deste sistema permitirá:

- Isolamento de dependências
- Facilidade de implantação
- Escalabilidade horizontal
- Portabilidade entre ambientes
- Padronização do ambiente de execução

Arquitetura de Containers

```
flowchart TD
    subgraph "AWS Cloud"
        subgraph "Amazon ECS/EKS Cluster"
            A[Frontend Container]
            B[API Gateway Container]
            C[Backend Core Container]
            D[Model Container]
            E[Redis Cache Container]
            F[Database Container]
        end
        G[Amazon S3]
        H[Amazon CloudFront]
        I[AWS Secrets Manager]
        J[Amazon CloudWatch]
    end
```

```

A <-->|HTTP/SSE| B
B <-->|REST| C
C <-->|gRPC| D
C <-->|Redis Protocol| E
C <-->|SQL/NoSQL| F
A <-->|Assets| G
H <-->|CDN| G
I <-->|Secrets| B
I <-->|Secrets| C
I <-->|Secrets| D
J <-->|Logs/Metrics| A
J <-->|Logs/Metrics| B
J <-->|Logs/Metrics| C
J <-->|Logs/Metrics| D
J <-->|Logs/Metrics| E
J <-->|Logs/Metrics| F
end

K[Usuário] <-->|HTTPS| H
K <-->|HTTPS| A

```

Containers Principais

1. Frontend Container

- Aplicação web para interação com usuários
- Implementa interface do chat
- Gerencia conexões SSE com o backend
- Tecnologias: React/Vue/Angular, Node.js

2. API Gateway Container

- Gerencia roteamento de requisições
- Implementa autenticação e autorização
- Controla limites de requisições (rate limiting)
- Tecnologias: Express.js, Kong, ou AWS API Gateway como serviço gerenciado

3. Backend Core Container

- Implementa a lógica de negócios principal
- Coordena fluxo entre componentes
- Gerencia sessões e estados de chat
- Tecnologias: Node.js, Express, Socket.io ou implementação SSE personalizada

4. Model Container

- Executa o modelo de linguagem
- Processa consultas e gera respostas
- Implementa validação de schemas

- Tecnologias: Python com frameworks como TensorFlow, PyTorch, ou integração com modelos hospedados

5. Redis Cache Container

- Armazena dados em cache para rápido acesso
- Gerencia filas de mensagens
- Mantém estados temporários
- Tecnologia: Redis

6. Database Container

- Armazena dados persistentes
- Mantém histórico de conversas
- Armazena informações de usuários
- Tecnologias: PostgreSQL, MongoDB, ou DynamoDB

Serviços AWS Recomendados

1. Amazon ECS (Elastic Container Service) ou Amazon EKS (Elastic Kubernetes Service)

- Orquestração de containers
- Gerenciamento de escalabilidade
- Balanceamento de carga
- Integração com outros serviços AWS

2. Amazon ECR (Elastic Container Registry)

- Armazenamento seguro de imagens Docker
- Integração com CI/CD
- Escaneamento de vulnerabilidades

3. AWS Fargate

- Execução serverless de containers
- Elimina necessidade de gerenciar servidores
- Pagamento apenas por recursos utilizados

4. Amazon S3

- Armazenamento de assets estáticos
- Backup de dados
- Armazenamento de logs

5. Amazon CloudFront

- CDN para distribuição global de conteúdo
- Redução de latência
- Proteção contra DDoS

6. Amazon RDS ou Amazon DynamoDB

- Banco de dados gerenciado
- Backup automático
- Alta disponibilidade

7. Amazon ElastiCache

- Redis gerenciado
- Replicação automática
- Escalabilidade simplificada

8. Amazon CloudWatch

- Monitoramento e logs
- Alertas
- Dashboards

9. AWS Secrets Manager

- Gerenciamento seguro de credenciais
- Rotação automática de secrets
- Integração com IAM

10. Amazon SageMaker (opcional)

- Hospedagem de modelos ML de forma gerenciada
- Inferência em tempo real
- Monitoramento de performance

Configuração dos Containers

Frontend Container (Dockerfile)

```
FROM node:18-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

FROM nginx:alpine
COPY --from=build /app/build /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Backend Core Container (Dockerfile)

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
ENV NODE_ENV=production
EXPOSE 3001
CMD ["node", "src/server.js"]
```

Model Container (Dockerfile)

```
FROM python:3.10-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
ENV MODEL_PATH=/app/models
EXPOSE 5000
CMD ["python", "src/model_server.py"]
```

Redis Cache Container

Utilizar imagem oficial do Redis:

```
FROM redis:7-alpine
COPY redis.conf /usr/local/etc/redis/redis.conf
CMD ["redis-server", "/usr/local/etc/redis/redis.conf"]
```

Database Container

Recomendado usar serviços gerenciados da AWS como RDS ou DynamoDB, mas caso seja necessário container:

```
FROM postgres:15-alpine
ENV POSTGRES_PASSWORD=password
ENV POSTGRES_USER=user
ENV POSTGRES_DB=chatdb
COPY init.sql /docker-entrypoint-initdb.d/
EXPOSE 5432
```

Fluxo de Comunicação

```
sequenceDiagram
    participant User as Usuário
    participant FC as Frontend Container
    participant AG as API Gateway
    participant BC as Backend Core
    participant MC as Model Container
    participant RC as Redis Cache
    participant DB as Database

    User->>FC: Envia consulta
    FC->>AG: Encaminha requisição
    AG->>BC: Processa requisição
    BC->>RC: Verifica cache
    alt Cache hit
        RC-->>BC: Retorna resposta em cache
    else Cache miss
        BC->>MC: Solicita processamento
        MC->>BC: Retorna resposta
        BC->>RC: Armazena em cache
    end
    BC->>DB: Registra interação
    BC-->>AG: Retorna resposta via SSE
    AG-->>FC: Transmite eventos SSE
    FC-->>User: Apresenta resposta
```

Considerações de Segurança

1. Segurança de Imagens

- Escaneamento de vulnerabilidades no ECR
- Utilização de imagens base oficiais e atualizadas
- Minimização do tamanho das imagens

2. Configuração de Rede

- Utilização de VPC para isolamento
- Security Groups para controle de acesso
- Configuração de WAF para proteção de API

3. Gerenciamento de Credenciais

- AWS Secrets Manager para armazenamento de secrets
- IAM Roles para autenticação entre serviços
- Não inclusão de credenciais em imagens Docker

4. Monitoramento e Logs

- Centralização de logs no CloudWatch
- Alertas para comportamentos suspeitos
- Monitoramento de performance

Plano de Implementação

Fase 1: Desenvolvimento e Testes Locais

1. Criar Dockerfiles para cada componente
2. Implementar docker-compose.yml para ambiente de desenvolvimento
3. Testar comunicação entre containers localmente
4. Implementar testes de integração

Fase 2: Configuração da Infraestrutura AWS

1. Configurar VPC, subnets e security groups
2. Provisionar ECR para armazenar imagens
3. Configurar ECS/EKS cluster
4. Implementar serviços gerenciados (RDS, ElastiCache)

Fase 3: CI/CD e Implantação

1. Configurar pipelines de CI/CD (AWS CodePipeline, GitHub Actions)
2. Implementar estratégia de deploy (Blue/Green, Canary)
3. Configurar monitoramento e alertas
4. Implementar backups e estratégia de recuperação de desastres

Fase 4: Otimização e Escalabilidade

1. Configurar auto-scaling baseado em métricas
2. Otimizar custos com instâncias spot (quando aplicável)
3. Implementar balanceamento de carga geográfico
4. Otimizar performance e custos

Exemplo de docker-compose.yml para Desenvolvimento

```
version: "3.8"

services:
  frontend:
    build:
      context: ./frontend
    ports:
      - "80:80"
    depends_on:
      - api-gateway
    environment:
      - API_URL=http://api-gateway:3001

  api-gateway:
    build:
      context: ./api-gateway
    ports:
```

```
    - "3001:3001"
  depends_on:
    - backend
  environment:
    - BACKEND_URL=http://backend:4000
    - JWT_SECRET=dev-secret

backend:
  build:
    context: ./backend
  ports:
    - "4000:4000"
  depends_on:
    - model
    - redis
    - postgres
  environment:
    - MODEL_URL=http://model:5000
    - REDIS_URL=redis://redis:6378
    - DATABASE_URL=postgres://user:password@postgres:5432/chatdb
    - NODE_ENV=development

model:
  build:
    context: ./model
  ports:
    - "5000:5000"
  volumes:
    - ./model/data:/app/data
  environment:
    - MODEL_PATH=/app/data/model
    - MAX_TOKENS=4096

redis:
  image: redis:7-alpine
  ports:
    - "6378:6378"
  volumes:
    - redis-data:/data

postgres:
  image: postgres:15-alpine
  ports:
    - "5432:5432"
  environment:
    - POSTGRES_PASSWORD=password
    - POSTGRES_USER=user
    - POSTGRES_DB=chatdb
  volumes:
    - postgres-data:/var/lib/postgresql/data

volumes:
```

```
redis-data:  
postgres-data:
```

Para implantar este sistema na AWS com a modelagem proposta, recomendamos começar definindo a infraestrutura como código utilizando AWS CDK, Terraform ou CloudFormation para garantir a consistência e reprodutibilidade da implantação.